Adaptive Traffic Light Controller with Reinforcement Learning for Reducing Traffic Congestion

Anisa Dwizarah*, Jalu Reswara, Infall Syafalni, Nana Sutisna, and Trio Adiono
School of Electrical Engineering and Informatics, Bandung Institute of Technology, Indonesia
University Center of Excellence on Microelectronics, Bandung Institute of Technology, Indonesia
*Email: 23223005@mahasiswa.itb.ac.id

***Abstract*—Reinforcement Learning is a machine learning method that has been adopted in smart traffic lights to reduce traffic congestion effectively. The algorithm is employed as an agent, like a traffic police officer, that can manage traffic jams with the intelligent capability. In this paper, we revisit the basic Q-Learning method applied to adaptive traffic lights by proposing systematic state-action definition, formulating reward policy, and evaluating the performance. The proposed method basically aims to adjust the duration of green lights at each intersection according to the level of congestion at that intersection dynamically, hence, can adapt with traffic condition. Simulation results confirm that the propose method works effectively in managing traffic congestion by maintaining vehicle queue length almost constant at around 40 vehicles queue, offering an improvement from the conventional method that give growth in queue more than 60 vehicles. In the future, we plan to implement the proposed work in an embedded hardware systems to address the real-time performance requirement.**

Keywords: reinforcement learning, traffic light control, Q-learning, green light duration

I. Introduction

Congestion is a common problem faced by every country worldwide, especially in densely populated urban areas with high mobility demands. In urban areas with a greater number of vehicles and a need for high mobility, congestion becomes even more severe, posing significant challenges to the rapid mobility needs of metropolitan cities. Indonesia is among the countries with severe congestion, particularly in major cities like the capital, Jakarta [1][2].

The remainder of this paper is organized as follows: Section II reviews related work in the field and various references from other researchers' work that have been adapted and applied in this paper itself, with each referenced work being written and indexed in the reference section at the end of the paper. Section III explains the methodology, detailing the steps used in this paper and how the process and flow lead to the results and solutions to the issues addressed. Section IV demonstrates the implementation of the paper in its application, whether in software, hardware, or simulation. Section V presents the experimental results and further discusses aspects that could still be developed from this paper. Section VI outlines the challenges and limitations of this work and describes the future work the author will undertake, continuing from this paper. Section VII concludes this paper with a discussion of future research and the potential expansion of this study. Finally, Section VIII provides the final conclusion of this paper, presenting the ultimate solutions obtained based on the problems raised in the introduction. Last but not least, a reference list is included as an acknowledgment of the researchers whose work has inspired the author of this paper.

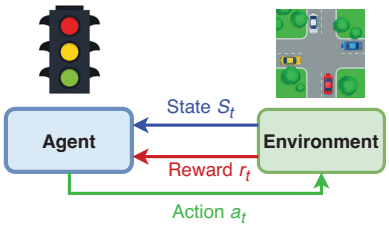


Figure 1. Reinforcement Learning Flow.

II. Related Work

The paper "Q-Learning" by Watkins and Dayan discusses the convergence and theoretical underpinnings of Q-learning, a model-free reinforcement learning algorithm. Q-learning allows an agent to learn how to act optimally in controlled Markovian environments through experience, without needing a model of the environment. It incrementally improves the estimates of the quality of actions taken in specific states based on rewards received, leading to the discovery of an optimal policy [3]. Q-Learning is employed as the subsequent step in determining the optimal decisions to be

TENCON 2024 - 2024 IEEE Region 10 Conference (TENCON) | 979-8-3503-5082-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/TENCON61640.2024.10902706

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 06:59:30 UTC from IEEE Xplore. Restrictions apply.

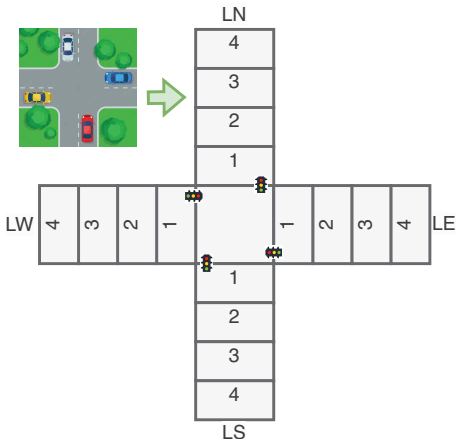


Figure 2. Congestion Level.

made by the agent [3][4]. Through the following formula, the Q-value in the Q-matrix $Q(s, a)$ is updated by:

$$Q(s,a) + \alpha \left[r + \gamma \max a' Q(s', a') - Q(s,a) \right], \quad (1)$$

where $Q(s, a)$ is the quality of action a in state s, α is the learning rate, r is the reward received, γ is the discount factor, and $\max a' Q(s', a')$ is the maximal expected future reward given new state s' and all possible actions a'.

III. Methodology

A. State and Action Definition

In this paper, the term *State* is defined as the current condition of the road environment, which is monitored in real-time. Initially, it is imperative to understand that the *Environment* referred to herein is conceptualized as a road intersection encompassing four converging pathways: the north (**LN**), east (**LE**), south (**LS**), and west (**LW**) routes. Each of these routes is further subdivided into four congestion levels—Level **1**, Level **2**, Level **3**, and Level **4**, as illustrated in Figure 2.

Then, the "*Initial Value*" on the matrix below will used to make different "*State*" between each "*Line*" (North, East, South, West):

Line Name	**Initial Value**
Line North	2^0
Line East	2^1
Line South	2^2
Line West	2^3

Now we have "*Initial Value*" for each line. Example, when the formula shown 2^0, we can recognize it means we now work on **Line North**. Next, we need to make initial for each level on the line too, using the Initial Value from those line, we first use this formula below to give all the level "*Initial*":

Line	**Initial**	**Formulation**
LN	2^0	$2^0 \times$ Congestion Level
LE	2^1	$2^1 \times$ Congestion Level
LS	2^2	$2^2 \times$ Congestion Level
LW	2^3	$2^3 \times$ Congestion Level

Employing the rules established in the preceding matrix, we will subsequently proceed to multiply each congestion level $1, 2, 3, 4$ for the respective roads by 2^0, 2^1, 2^2, and 2^3, corresponding to the line on which the congestion occurs. The product of these multiplications will yield outcomes as depicted in the following matrix:

Congestion Level	**1**	**2**	**3**	**4**
LN	1	2	3	4
LE	2	4	6	8
LS	4	8	12	16
LW	8	16	24	32

To facilitate comprehension, we may convert this into a list as follows. In this enumeration, each congestion level has been transformed into symbols representing distinct values. For instance, the value 4 on the LN route denotes the fourth level of congestion (100 vehicles on the road), whereas on the LE route, a value of 4 signifies the second level of congestion (50 vehicles on the road). Conversely, for the LS route, the value 4 represents only the first level of congestion (25 vehicles on the road). It is imperative to interpret each value in the context of the list in which it appears; despite identical numerical representations, the values denote varying levels of congestion depending on the route they correspond to.

- LN = [1, 2, 3, 4]
- LE = [2, 4, 6, 8]
- LS = [4, 8, 12, 16]
- LW = [8, 16, 24, 32]

It is imperative to delineate the definition of each congestion level, which is as follows:

- **Level 1**: 25 vehicles on the road.
- **Level 2**: 50 vehicles on the road.
- **Level 3**: 75 vehicles on the road.
- **Level 4**: 100 vehicles on the road.

Next, the congestion values identified by the environment for each path at the intersection are input into the subsequent formula [5]:

$$(2^0 \times LN) + (2^1 \times LE) + (2^2 \times LS) + (2^3 \times LW) \quad (2)$$

As a result, there will be 256 unique outcomes based on the congestion level inputs provided, yielding values

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 06:59:30 UTC from IEEE Xplore. Restrictions apply.

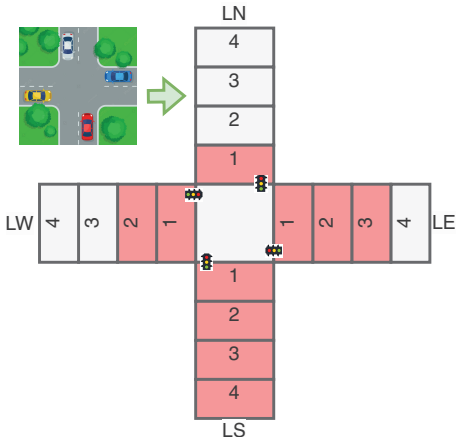


Figure 3. Congestion Level Illustration.

ranging from 85 to 340. This allows us to define 256 distinct *states*, each representing different traffic congestion conditions. Through this formulation, each traffic condition on the road can be quantified into a value that is computable using mathematical algorithms. Let us consider an example corresponding to Figure 3, which delineates the following levels of congestion:

- Congestion level for Line North: **1**
- Congestion level for Line East: **3**
- Congestion level for Line South: **4**
- Congestion level for Line West: **2**

These congestion levels are then converted into numerical values according to the matrix table, resulting in:

- LN = 1
- LE = 6
- LS = 16
- LW = 16

Subsequently, these values for LN, LE, LS, and LW are integrated into the aforementioned formula, culminating in the following calculation:

$$(2^0 \times 1) + (2^1 \times 6) + (2^2 \times 16) + (2^3 \times 16) \quad (3)$$

The computation process for the congestion values is shown step-by-step below:

$$\textit{State} = (1 \times 1) + (2 \times 6) + (4 \times 16) + (8 \times 16) \quad (4)$$

$$\textit{State} = 1 + 12 + 64 + 128 = 205$$

$$\textit{State} = 205$$

In the Q-matrix representation, one will notice that the action columns are initially unpopulated. There are four such

State	a_1(LN)	a_2(LE)	a_3(LS)	a_4(LW)
S_1	$Q_{a1}[1]$	$Q_{a2}[1]$	$Q_{a3}[1]$	$Q_{a4}[1]$
S_2	$Q_{a1}[2]$	$Q_{a2}[2]$	$Q_{a3}[2]$	$Q_{a4}[2]$
S_3	$Q_{a1}[3]$	$Q_{a2}[3]$	$Q_{a3}[3]$	$Q_{a4}[3]$
...	...	...	...	...
S_{256}	$Q_{a1}[256]$	$Q_{a2}[256]$	$Q_{a3}[256]$	$Q_{a4}[256]$

Figure 4. Q-Matrix Illustration.

action columns designated as a1 (LN), a2 (LE), a3 (LS), and a4 (LW). Each of these columns will eventually be populated with distinct values, which are described as follows:

- 20 = increase duration by 20 seconds
- 40 = increase duration by 40 seconds
- 60 = increase duration by 60 seconds
- 80 = increase duration by 80 seconds

The action consists of the activation of traffic lights on the currently serviced line α = action

a_1 **(LN)** = Green light at North
a_2 **(LE)** = Green light at East
a_3 **(LS)** = Green light at South
a_4 **(LW)** = Green light at West

Drawing from the previous discussion, consider the condition of *state* number **205**, which indicates the level of congestion on each respective line at that time. Given such congestion, the values of the green light duration to be assigned to the lanes would be as shown in table below:

B. Reward Definition

For the *reward* metric, it is derived from the observed on-site conditions following the initial *action* . If congestion decreases after the first *action* , the reward value will be higher. Conversely, if congestion does not improve or worsens, the received *reward* value will diminish. The *reward* value is calculated using the following formula:

Let r represent the *reward* , then the *reward* function $R(s, a)$ is given by:

$$R(s, a) = 1 - \frac{N(s')}{F} \quad (5)$$

Where:

- $N(s')$ is the number of cars on the road at *state* s' after executing an *action* and the initial *state* .
- F is the normalization factor, defined as the maximum number of vehicles that can be on the road for the calculation. If there are more vehicles, this value will be capped at the maximum limit, assumed to be 100.

For instance:

- If $N(s') = 0$, then $R(s, a) = 1$.
- If $N(s') = 20$, then $R(s, a) = 1 - \frac{20}{100} = 0.8$.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 06:59:30 UTC from IEEE Xplore. Restrictions apply.

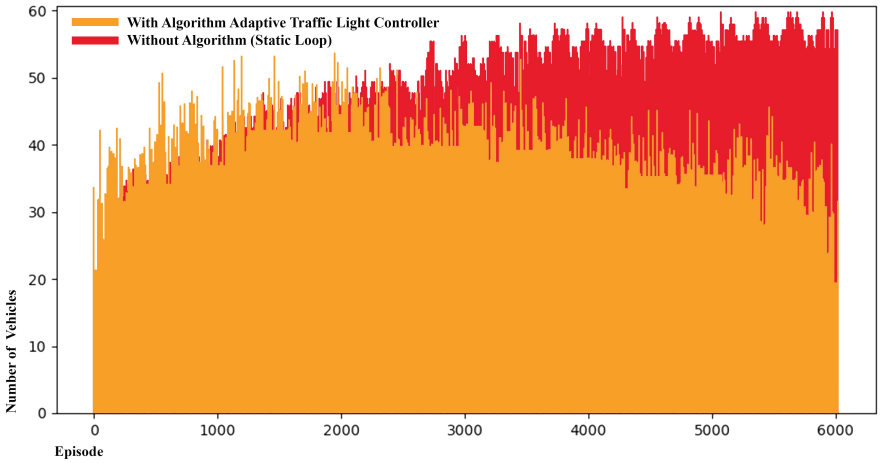


Figure 5. Total Vehicles Count in Simulation.

IV. Results and Discussion

In Figure 5, the graph shows the number of vehicles on the road at each simulation step. During the first 500 episodes, the system managed to maintain an average of 45 vehicles on the road. However, this number continued to rise significantly without being reduced, and by step 3000, the number of vehicles on the road kept increasing significantly without any reduction. This situation would persist, and as can be seen even in real-world scenarios, this condition will not be resolved until traffic police or a similar authority intervenes to address the problem.

In this study, up to 6000 episodes were conducted in Figure 5. Each episode involved collecting state data once, determining the best action, obtaining a reward value, calculating the Bellman equation, and finally recording the best value in the Q-Matrix. During each episode, the number of vehicles detected within the roadway scope was counted and the data stored. This data was then collected and consolidated into a graph showing the total number of vehicles. It can be observed that the average number of vehicles was 48 under normal conditions up to 2000 episodes, and it could be reduced to 32 vehicles, indicating an estimated traffic congestion reduction of 33

V. Conclusion and Future Work

The experiment to reduce traffic congestion using reinforcement learning methods will next be applied to a compact miniature traffic model with an embedded system. In this model, the values of factors affecting traffic congestion can be adjusted, and various simulation scenarios can be run. This approach is expected to facilitate traffic simulations with various road obstacles, such as sudden heavy traffic in one lane or the sudden complete closure of a lane. These unusual situations can later be implemented in a real simulator, making it easier for anyone to test it. With the compact embedded system, this approach is expected to be more streamlined and ergonomic.

References

[1] N. Sutisna, Z. N. Arifuzzaki, I. Syafalni, R. Mulyawan, and T. Adiono, "Architecture design of q-learning accelerator for intelligent traffic control system," in *2022 International Symposium on Electronics and Smart Devices (ISESD)*, Bandung, Indonesia, 2022, pp. 1–6. DOI: 10.1109/ISESD56103.2022.9980698.

[2] N. Sutisna *et al.*, "Deep q-network model for intelligent traffic light," in *2022 International Symposium on Electronics and Smart Devices (ISESD)*, Bandung, Indonesia, 2022, pp. 1–5. DOI: 10.1109/ISESD56103.2022.9980739.

[3] C. J. C. H. Watkins, "Learning from delayed rewards," [Online]. Available: http://www.cs.rhul.ac.uk/chrisw/new_thesis . pdf, Ph.D. dissertation, King's College, Cambridge, Cambridge, UK, May 1989.

[4] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[5] J. Reswara, N. Sutisna, I. Syafalni, and T. Adiono, "Q-learning algorithm with double-agent reinforcement learning for smart traffic controller," in *2023 IEEE 66th International Midwest Symposium on Circuits and Systems (MWSCAS)*, 2023, pp. 649–653. DOI: 10.1109/MWSCAS57524.2023.10406051.

Authorized licensed use limited to: VIT University. Downloaded on August 15,2025 at 06:59:30 UTC from IEEE Xplore. Restrictions apply.